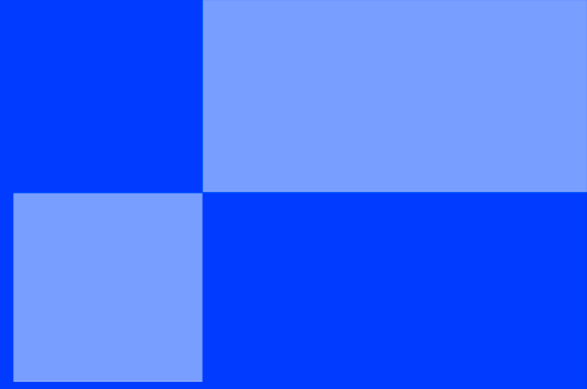


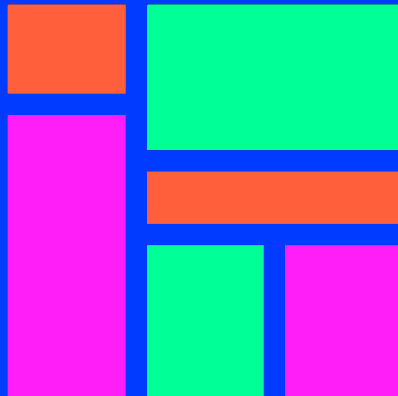
{EPITECH}



BOOTSTRAP



< DECISION MATRIX & REST API />



BOOTSTRAP

The Decision Matrix

Let's apply a structured, bias-resistant decision-making process to select both a backend language and a frontend framework suited to the specific constraints of your project.

First, define and lock your criteria: list 5 to 8 criteria that genuinely matter for your project. Consider:

- ✓ Team's existing experience with the technology;
- ✓ Ecosystem quality: available packages, third-party integrations, widget libraries;
- ✓ Suitability for real-time data refresh;
- ✓ Docker deployment complexity;
- ✓ Community support and long-term viability;
- ✓ Learning value for your own development goals.

For each criterion, assign a weight : 1 = nice to have, 2 = important, 3 = critical.
Together, you have to choose at least 3 options.

Then, each team member fills in the matrix **independently, without discussing with others**. Score each option from 1 (poor fit) to 5 (excellent fit) per criterion. Compute the weighted total for each option. It should look like this

Criterion	Weight	Node.js	Python	Java	React	Vue.js	Angular
[Criterion 1]	1						
[Criterion 2]	3						
...							
Weighted total							

After filling independently, compare your matrices with your teammates. Note any criteria where your scores diverge by more than 2 points.

Write a recommendation

Write a 250 to 300 word recommendation addressing:

- ✓ Your chosen backend language and frontend framework
- ✓ The top two criteria that drove your decision
- ✓ One significant trade-off you are accepting and how you will mitigate it
- ✓ Any point of disagreement within your team and how you resolved it

REST API Design

Let's build a basic HTTP server from scratch and expose some structured data through its routes.

A data store in JSON

Using the language of your choice, build an application entry point that initializes and starts an HTTP server on port 8000 (for instance `app.py` or `Main.java`). Seed it with a JSON file containing 3 hardcoded books:

```
[
  { id: 1, title: 'The Pragmatic Programmer', author: 'Andrew Hunt' },
  { id: 2, title: 'Clean Code', author: 'Robert C. Martin' },
  { id: 3, title: 'Design Patterns', author: 'GoF' }
]
```

Then, implement two basic routes:

- ✓ `GET /books` returns all books as a JSON array;
- ✓ `POST /books` with a JSON body `{"title": "...", "author": "..."}` adds a new book and returns the created object including its generated `id`.

Now, verify it's working properly with commands like:

- ✓ `curl http://localhost:8000/books`
- ✓ `curl -X POST http://localhost:8000/books -H "Content-Type: application/json" -d '{"title":"Déchaîner l'intelligence","author":"Pierre Robert"}'`

Asking for something that does not exist

Add two more routes to your server:

- ✓ `GET /books/:id` -- returns a single book matching the given `id`
- ✓ `DELETE /books/:id` -- removes the book with that `id`

For both routes, if no book matches the `id`, return HTTP status code 404 and:

```
{"error": "Book not found"}
```

Test `curl http://localhost:8000/books/9999`, it should not crash.

Who is allowed to write?

POST `/books` and DELETE `/books/:id` should only be accessible to authenticated requests. Require the header `Authorization: Bearer <token>` on those two routes.

For now, define one valid hardcoded token.

Expected behavior:

- ✓ Missing header -> HTTP 401
- ✓ Invalid token -> HTTP 403
- ✓ Valid token -> operation succeeds as before

GET `/books` and GET `/books/:id` remain public (no token required).

Authentication

You must be able to implement server-side authentication from scratch -- login form, session persistence, and third-party OAuth sign-in -- on a minimal standalone application.

A page only you can see

Create a project with two files: `index.html` (public) and `private.html` (private), and a server file (`server.js`, `app.py`, or equivalent) that runs on port 3000.

The server must handle the following routes:

- ✓ GET `/` -- serves `index.html` to anyone
- ✓ GET `/private` -- serves `private.html` only if the user is authenticated; otherwise redirects to `/login`
- ✓ GET `/login` -- serves a login form
- ✓ POST `/login` -- checks the submitted credentials against a single hardcoded user (`username: admin`, `password: s3cr3t`)

Open a new private window and go to `http://localhost:3000/private`.

You must land on `/login`, not on `private.html`.

Sign up, then stay logged in

Extend your `server.js` with two new routes:

- ✓ `POST /signup` -- creates a new user and stores it (in memory or in a `users.json` file)
- ✓ Remove the hardcoded `admin` user from the previous exercise

Passwords must not be stored in plain text -- choose how to protect them.

Then make the session survive a browser reload: log in, close the tab, reopen `http://localhost:3000/private`, and confirm you land on `private.html` without entering your password again.

Sign in with a third party

Add the following button to your login page:

```
<button id="oauth-github">Sign in with GitHub</button>
```

Clicking it should start an OAuth 2.0 flow with GitHub (or another provider of your choice). After a successful login, redirect the user to `/private`.

What happens when the email returned by the provider already exists in your `users.json`? You must handle this edge case.



Never store OAuth tokens in `users.json`. Think about what your server actually needs to keep.



Have you ever heard of `.env`?
And what about `.gitignore`?

External API Integration

You must be able to fetch live data from a public third-party API AND make the query configurable via a CLI argument, with a proper error handling.

What is the weather right now?

Create a file `weather.js` (or `weather.py` or `weather.????`) that calls a free public weather API and prints to the console the current temperature in any city of your choice, such as `Timbuktu: 42°C`

The city name must be stored in a variable at the top of the file -- not embedded directly in the URL string.

Cross-check the returned temperature against another weather source.



Several weather APIs have a free tier with no credit card required: Open-Meteo (no key needed), wttr.in (plain HTTP, no key), OpenWeatherMap (free plan).

Make it work for any city

Modify `weather.js` to accept a `--city` argument:

```
node weather.js --city London
```

Expected output: `London: 12°C`

Run with three different cities and confirm the output changes each time.

Running the script without `--city` should do something, but not crash!

When the service is down

Replace your API URL with `https://badurl.xyz/weather`, run the script and observe what happens. Then, add error handling so that the output becomes: `Error: could not fetch weather data (reason: <message>)`

The script must exit cleanly -- no uncaught exception, no stack trace shown to the user.

Restore the correct URL and verify the script still works normally.

Learn the fun way

If you are not comfortable with JavaScript and CSS, you can try some of the following games.

- ✓ [JSDares](#)
- ✓ [JSRobot](#)
- ✓ [Untrusted](#)
- ✓ [Elevator saga](#)
- ✓ [CSS diner](#)
- ✓ [Grid garden](#)
- ✓ [Grid Attack](#)
- ✓ [FlexBox froggy](#)
- ✓ [FlexBox defense](#)
- ✓ [FlexBox Adventure](#)



v 0.1

{EPITECH}